PhaseScreenZernike

November 15, 2021

[1]:
```
#imports
from aotools.turbulence import infinitephasescreen
from matplotlib import pyplot as plt
import numpy as np
from aotools import zernike
import pandas as pd
%matplotlib inline
```

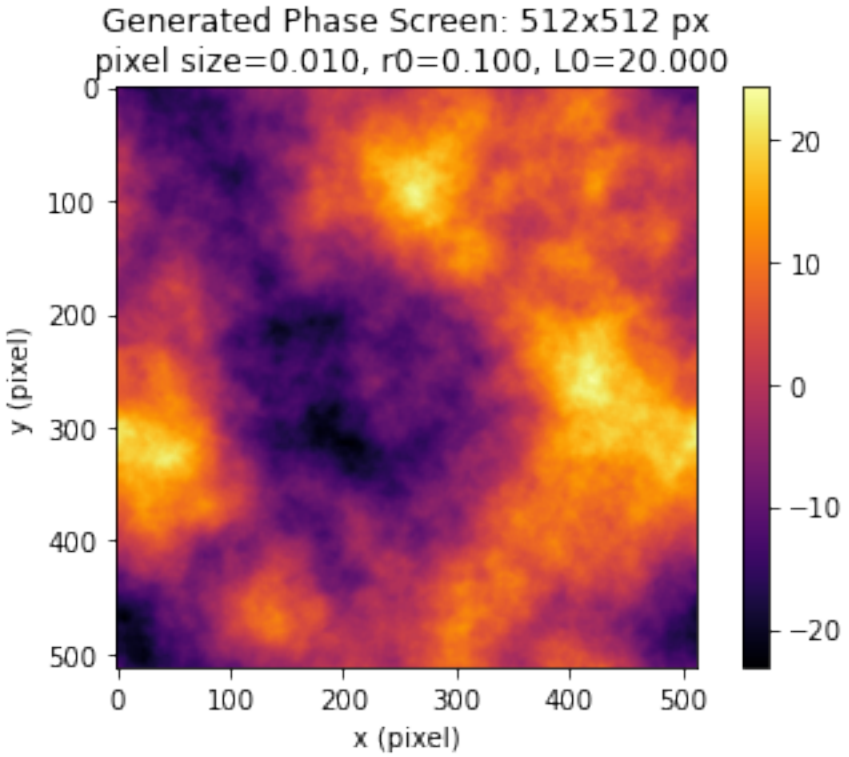
[2]:
```
#inputs
scrn_size=512
pixel_scale=0.01
r0=0.1
L0=20
radialDegree=10
#radialDegree    1   2   3    4   5   6   7   8   9  10  11  12  13  14  15  16␣
 ↪17  18  19  20
#zernike(approx) 1   3   6   10  15  21  28  36  45  55  66  78  91 105 120 136␣
 ↪153 171 190 210

#what should this class be able to do
#1 generate phase screen
#2 plot phase screen
#3 find zernike coeff
#4 plot zernike coeff
#5 plot fit phase

Nzernike=int(radialDegree*(radialDegree+1)/2)

phase_screen=infinitephasescreen.
 ↪PhaseScreenVonKarman(scrn_size,pixel_scale,r0,L0)#number of pixel, pixel size,␣
 ↪r0, L0
phase=phase_screen.scrn
plt.imshow(phase,cmap='inferno')
plt.xlabel('x (pixel)')
plt.ylabel('y (pixel)')
plt.title('Generated Phase Screen: {:d}x{:d} px\n pixel size={:.3f}, r0={:.3f},␣
 ↪L0={:.3f}'.format(scrn_size,scrn_size,pixel_scale,r0,L0))
```

```
plt.colorbar()
plt.show()
plt.clf()
```

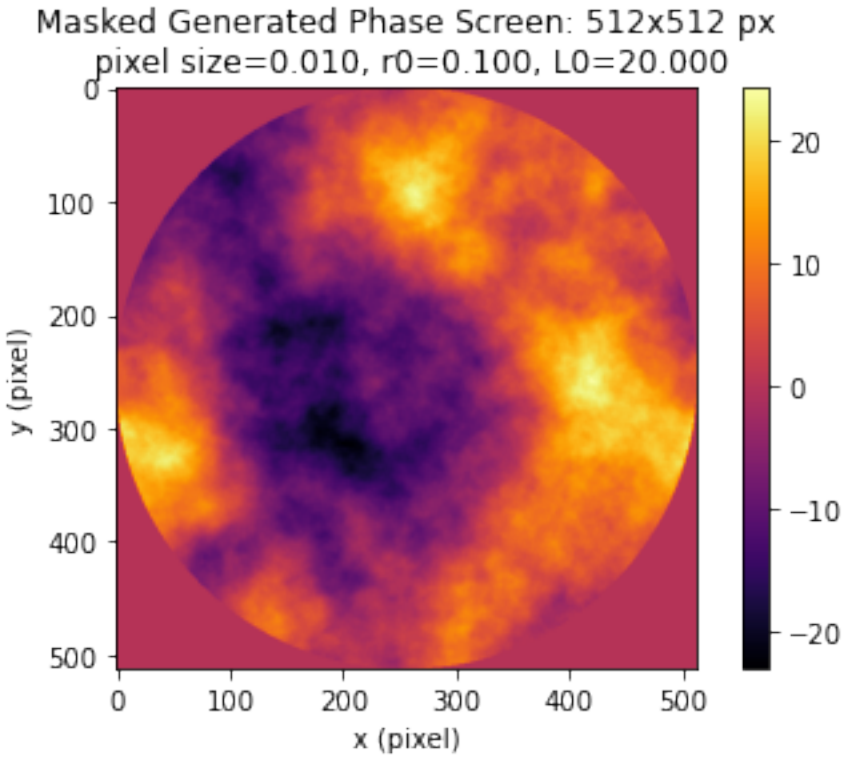



```
<Figure size 432x288 with 0 Axes>
```

[3]:
```
x=np.arange(scrn_size)
x=x-scrn_size/2
xx,yy=np.meshgrid(x,x)
r=np.sqrt(xx**2+yy**2)
mask=np.zeros(r.shape)
for i in range(r.shape[0]):
    for j in range(r.shape[1]):
        if r[i,j]<=scrn_size/2:
            mask[i,j]=1
        else:
            mask[i,j]=0

plt.imshow(mask*phase,cmap='inferno')
plt.xlabel('x (pixel)')
plt.ylabel('y (pixel)')
```

```
plt.title('Masked Generated Phase Screen: {:d}x{:d} px\n pixel size={:.3f}, r0={:
 ↪.3f}, L0={:.3f}'.format(scrn_size,scrn_size,pixel_scale,r0,L0))
plt.colorbar()
plt.show()
plt.clf()
```




```
<Figure size 432x288 with 0 Axes>
```

[4]:
```
modes=zernike.zernikeArray(Nzernike,scrn_size)
#for i in range(len(modes)):
#    plt.imshow(modes[i],cmap='inferno')
#    plt.xlabel('x (pixel)')
#    plt.ylabel('y (pixel)')
#    plt.title('Zernike Mode #{:d}'.format(i))
#    plt.colorbar()
#    plt.show()
#    plt.clf()
```

[5]:
```
normality_check=np.zeros(Nzernike)
zeromode=modes[0].sum()
for i in range(Nzernike):
```

```
    normality_check[i]=(modes[i]**2).sum()/(np.pi*((scrn_size/2)**2))

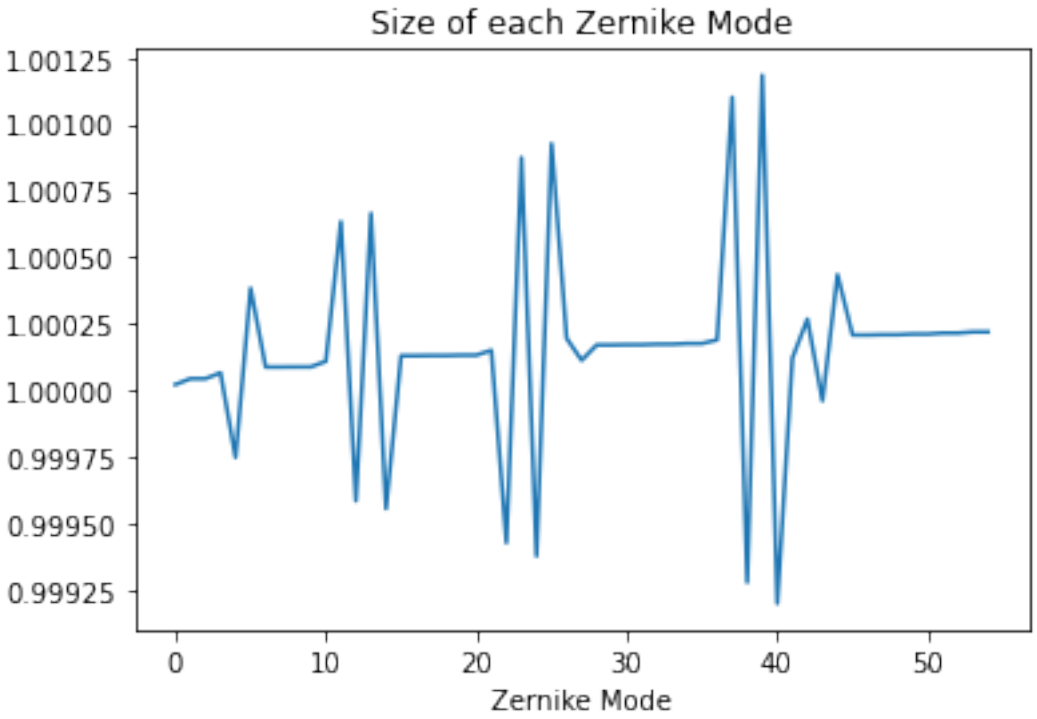
plt.plot(normality_check)
plt.xlabel('Zernike Mode')
plt.title('Size of each Zernike Mode')
plt.show()
plt.clf()

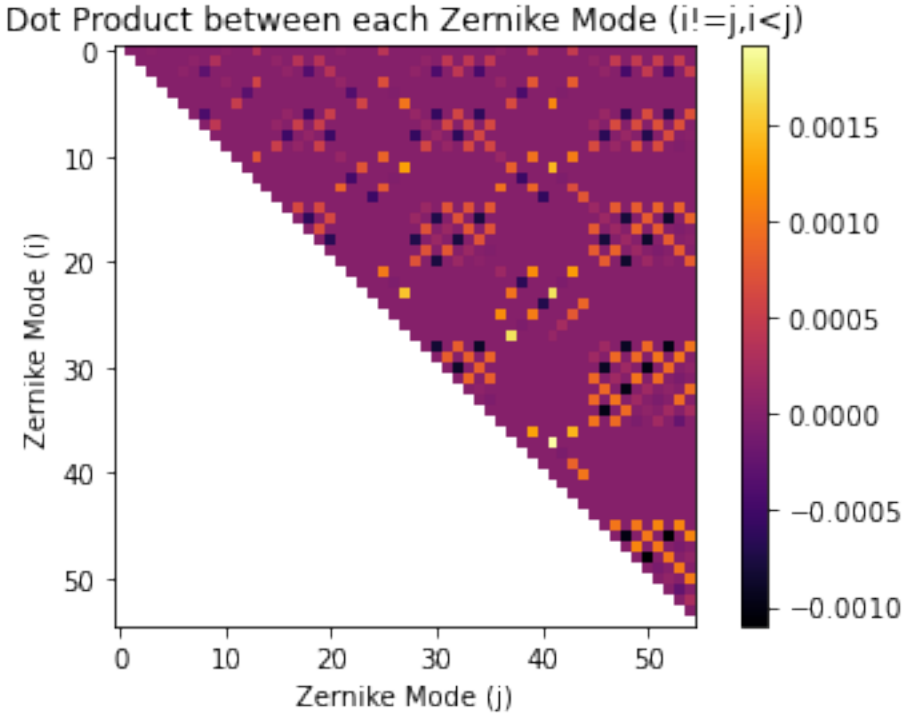
#print(normality_check)

orthogonality_check=np.empty((Nzernike,Nzernike))
orthogonality_check[:,:]=np.NaN
for i in range(Nzernike):
    for j in range(Nzernike):
        if i!=j:
            if i<j:
                orthogonality_check[i,j]=(modes[i]*modes[j]).sum()/((np.
 ↪pi*(scrn_size/2)**2)*normality_check[i]*normality_check[j])

plt.imshow(orthogonality_check,cmap='inferno')
plt.xlabel('Zernike Mode (j)')
plt.ylabel('Zernike Mode (i)')
plt.title('Dot Product between each Zernike Mode (i!=j,i<j)')
plt.colorbar()
plt.show()
plt.clf()

#pd.DataFrame(orthogonality_check)
```


Size of each Zernike Mode
1.00125
1.00100
1.00075
1.00050
1.00025
1.00000
0.99975
0.99950
0.99925
0
10
20
30
40
50
Zernike Mode


Dot Product between each Zernike Mode (i!=j,i<j)
Zernike Mode (i)
Zernike Mode (j)
0
10
20
30
40
50
0.0015
0.0010
0.0005
0.0000
−0.0005
−0.0010

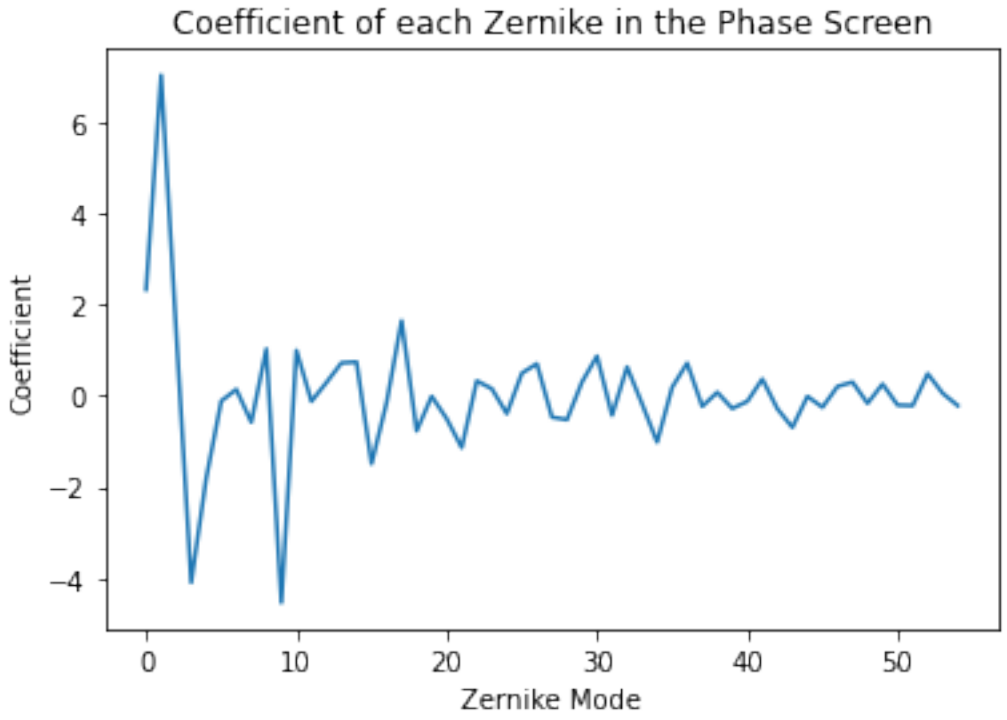
```
<Figure size 432x288 with 0 Axes>
```

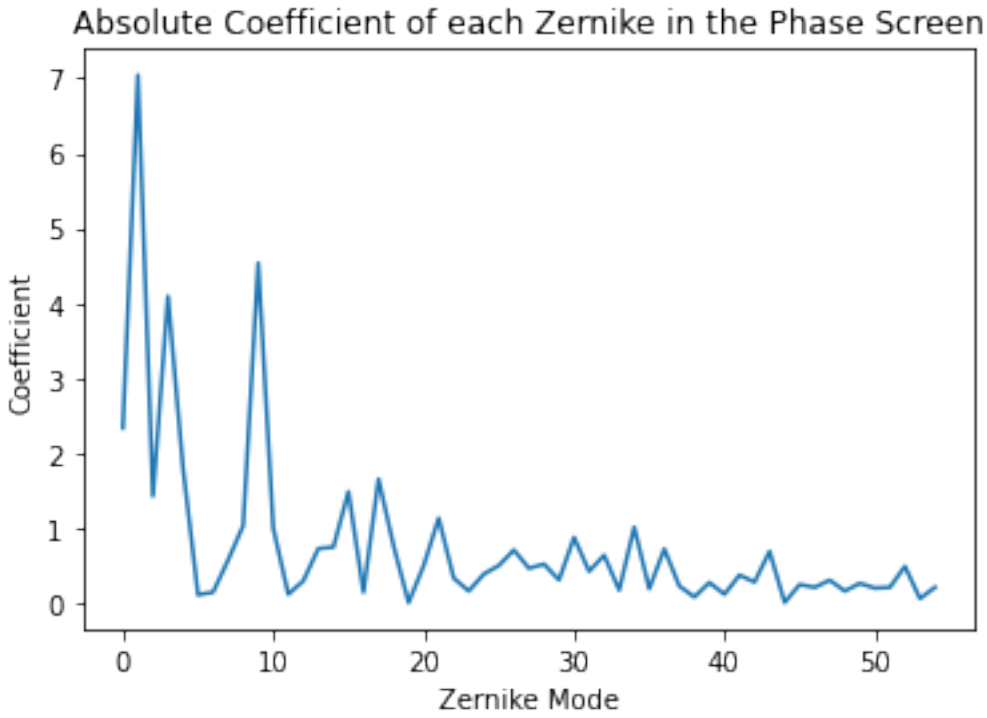
[6]:
```
phase_screen=infinitephasescreen.
 ↪PhaseScreenVonKarman(scrn_size,pixel_scale,r0,L0)#number of pixel, pixel size,␣
 ↪r0, L0
phase=phase_screen.scrn

coeff=np.zeros(Nzernike)
for i in range(Nzernike):
    coeff[i]=(mask*phase*modes[i]).sum()/(normality_check[i]*(np.pi*(scrn_size/
 ↪2)**2))

pd.DataFrame(coeff)
plt.plot(coeff)
plt.title('Coefficient of each Zernike in the Phase Screen')
plt.xlabel('Zernike Mode')
plt.ylabel('Coefficient')
plt.show()
plt.clf()

plt.plot(np.sqrt(coeff**2))
plt.title('Absolute Coefficient of each Zernike in the Phase Screen')
plt.xlabel('Zernike Mode')
plt.ylabel('Coefficient')
plt.show()
plt.clf()
```






```
<Figure size 432x288 with 0 Axes>
```

[7]:
```
observe=mask*phase
calculate=zernike.phaseFromZernikes(coeff,scrn_size)
residue=observe-calculate

mapmax=max([observe.max(),calculate.max(),residue.max()])
mapmin=min([observe.min(),calculate.min(),residue.min()])

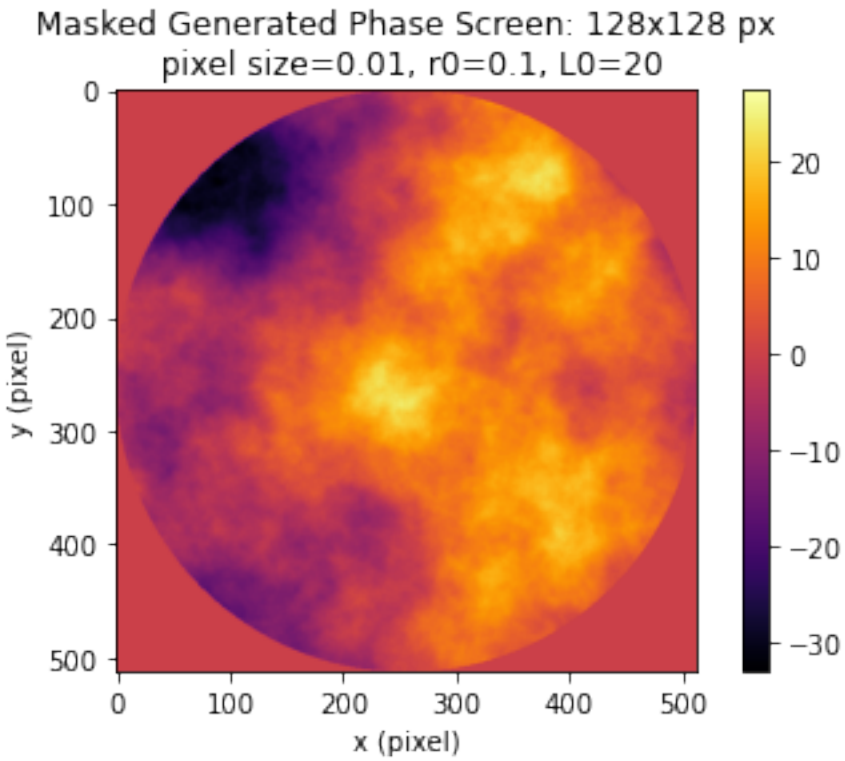
plt.imshow(observe,cmap='inferno',vmin=mapmin,vmax=mapmax)
plt.xlabel('x (pixel)')
plt.ylabel('y (pixel)')
plt.title('Masked Generated Phase Screen: 128x128 px\n pixel size=0.01, r0=0.1,␣
 ↪L0=20')
plt.colorbar()
plt.savefig('PhaseScreen')
plt.show()
plt.clf()

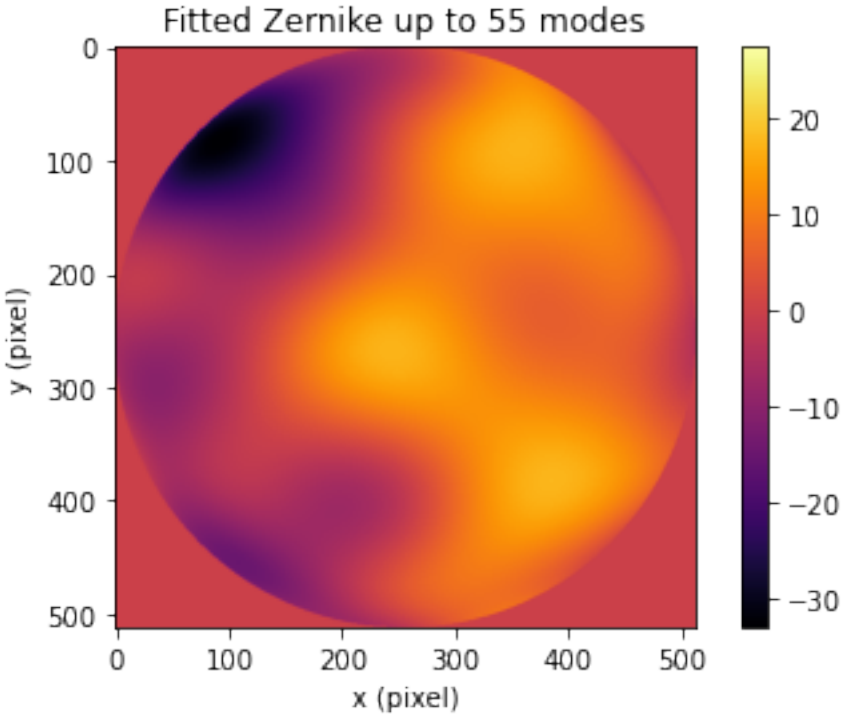
plt.imshow(calculate,cmap='inferno',vmin=mapmin,vmax=mapmax)
plt.colorbar()
```

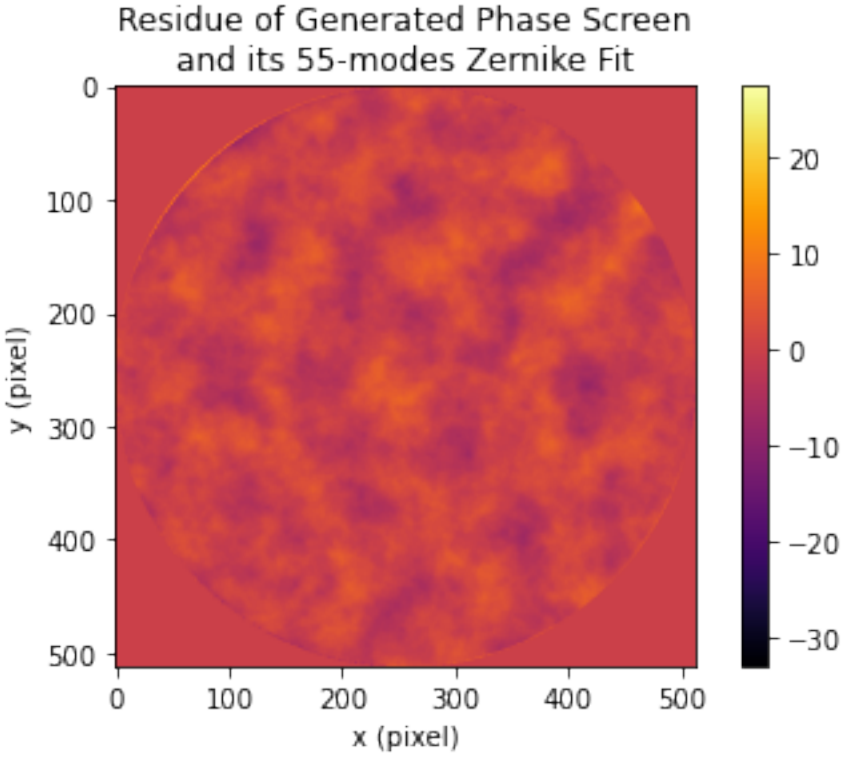
```
plt.title('Fitted Zernike up to {:d} modes'.format(Nzernike))
plt.xlabel('x (pixel)')
plt.ylabel('y (pixel)')
plt.savefig('FittedZernike')
plt.show()
plt.clf()


plt.imshow(residue,cmap='inferno',vmin=mapmin,vmax=mapmax)
plt.colorbar()
plt.title('Residue of Generated Phase Screen\nand its {:d}-modes Zernike Fit'.
 ↪format(Nzernike))
plt.xlabel('x (pixel)')
plt.ylabel('y (pixel)')
plt.savefig('Residue')
plt.show()
plt.clf()

print('observe rms=')
print(observe.std())
print('fit rms=')
print(calculate.std())
print('residue rms=')
print(residue.std())
```




Fitted Zernike up to 55 modes
y (pixel)
x (pixel)
0
100
200
300
400
500
20
10
0
−10
−20
−30


Residue of Generated Phase Screen
and its 55-modes Zernike Fit
y (pixel)
x (pixel)
0
100
200
300
400
500
20
10
0
−10
−20
−30

```
observe rms=
9.545941719453639
fit rms=
9.300085943201866
residue rms=
2.149749750330956
```

```
<Figure size 432x288 with 0 Axes>
```